

LECTURE

Introduction to HashiCorp Terraform





What is Terraform?



Created by HashiCorp

Often more famous by its tools rather than its name. It also offers many industry-standard solutions



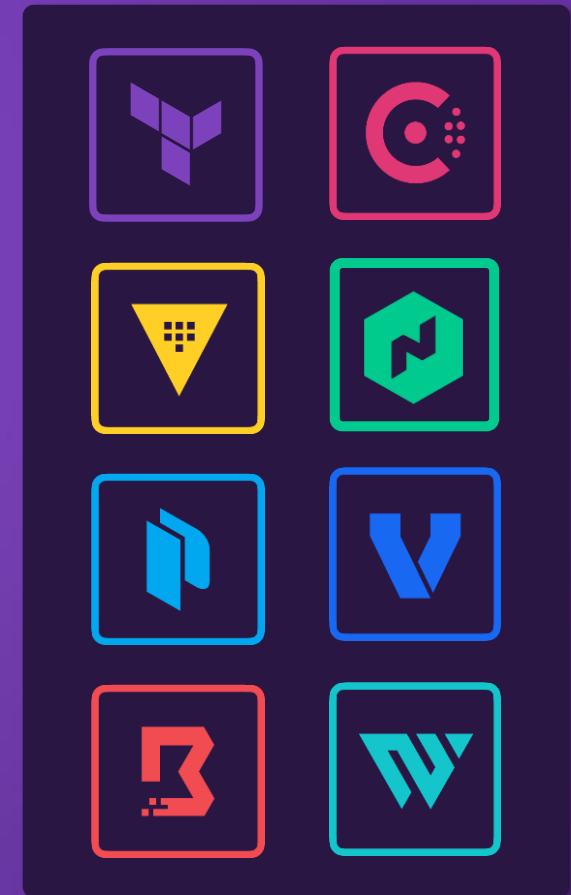
Declarative Approach

Focuses on describing "what" needs to be done rather than explicitly stating "how" to do it, allowing the system to determine the best execution steps



Based on HashiCorp Configuration Language (HCL)

A human-readable language used in Terraform to define infrastructure resources in a clear, structured, and reusable way





What is Terraform?

The industry standard Infrastructure as Code (IaC) tool that automates the provisioning, management, and versioning of infrastructure.



Codify Your Infrastructure

Rather than clicking around in a console, create a repeatable, versioned workflow to provision and manage resources



Standardize Workflows

Use a single tool to deploy and manage resources required for application workloads. Quickly scale up using modules and other features.



Platform Agnostic

Deploy infrastructure as code using HCL to provision resources from any cloud or datacenter.

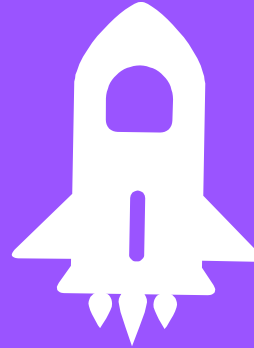


Benefits of Terraform



VERSION CONTROL AND AUDITIBILITY

Infrastructure changes are tracked in version control systems, providing a clear history and ability to revert



SCALABILITY & FLEXIBILITY

Quickly scale infrastructure up or down to adapt to changing requirements by modifying code instead of manually changing resources.



COLLABORATION

Teams can collaborate on infrastructure changes just like application code, improving transparency and reducing silos in operations



AUTOMATION AND EFFICIENCY

Automate the provisioning and management of infrastructure, reducing manual tasks and accelerating deployment times



CONSISTENCY & REPEATABILITY

Ensures that infrastructure is consistently deployed across environments, reducing configuration drift and human errors

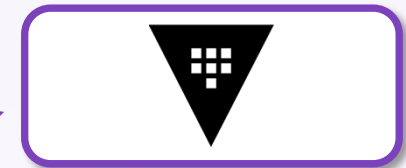
Terraform is Platform Agnostic



Cloud Platforms



Other Services



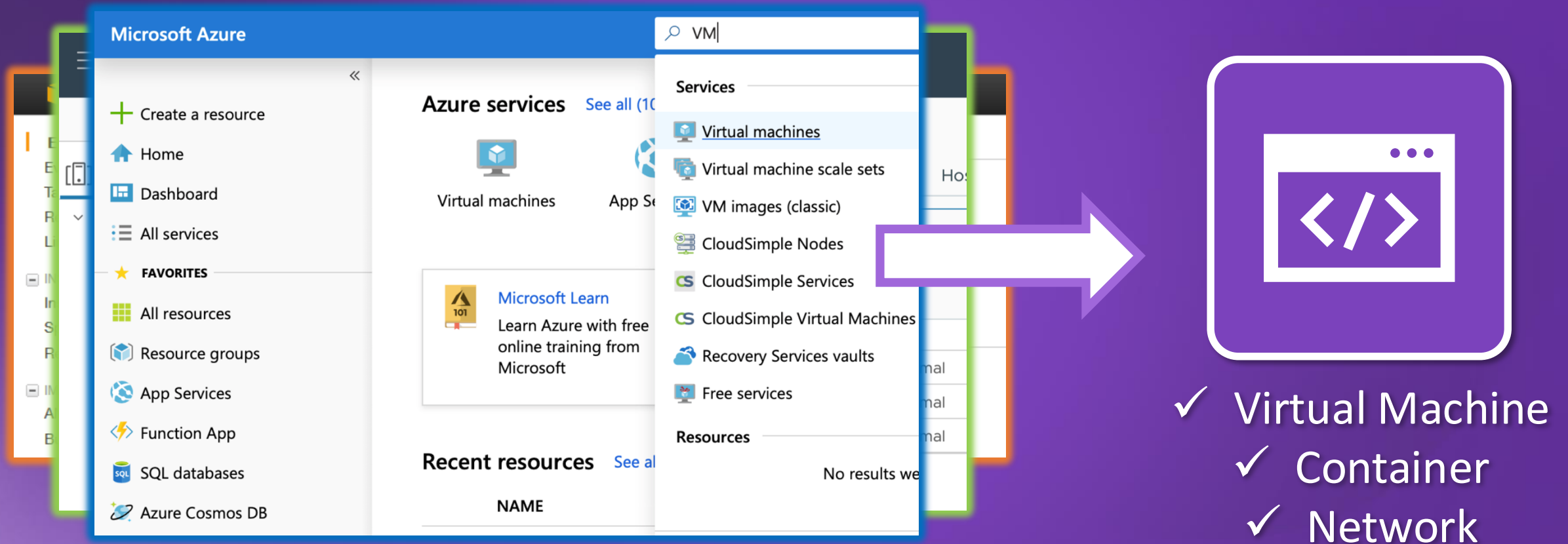


Before Using Terraform, You Need to First Be Proficient on the Target Platform

While it might feel like it, Terraform isn't magic. You need to understand the target platform before using Terraform.



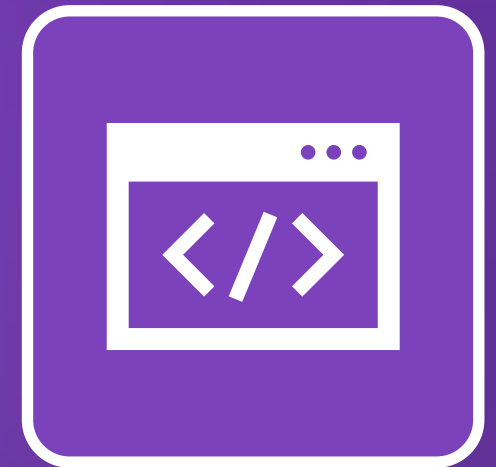
Traditional Process of Provisioning Resources



Provisioning with Terraform



```
1 resource "aws_instance" "ubuntu_server" {
2   ami           = data.aws_ami.ubuntu.id
3   instance_type = "t3.micro"
4   subnet_id     = aws_subnet.public_subnets["public_subnet_1"].id
5   security_groups = [aws_security_group.vpc-ping.id]
6   associate_public_ip_address = true
7   key_name       = aws_key_pair.generated.key_name
8   connection {
9     user          = "ubuntu"
10    private_key    = tls_private_key.generated.private_key_pem
11    host           = self.public_ip
12  }
}
```



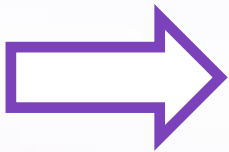
Application
Workload

Reusing Code with Terraform

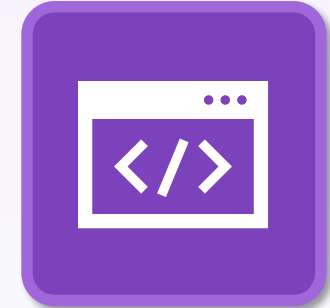


Input Variables

AMI = Ubuntu
Subnet = 10.16.7.0
App = Web Server



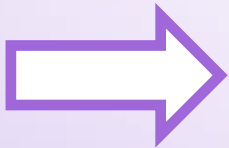
```
1 resource "aws_instance" "basic_server" {  
2   ami                = data.aws_ami.image.id  
3   instance_type      = var.instance_size  
4   subnet_id          = aws_subnet.public_subnets.id  
5   security_groups    = [aws_security_group.ingress-ssh.id]  
6   associate_public_ip_address = true  
7   key_name           = aws_key_pair.generated.key_name  
8 }
```



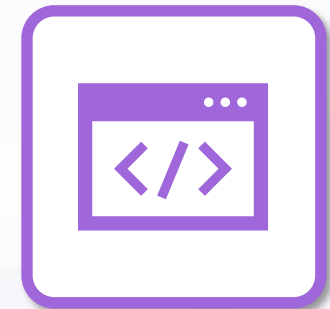
Web
Application

Input Variables

AMI = Windows
Subnet = 10.0.3.0
App = Data Processing



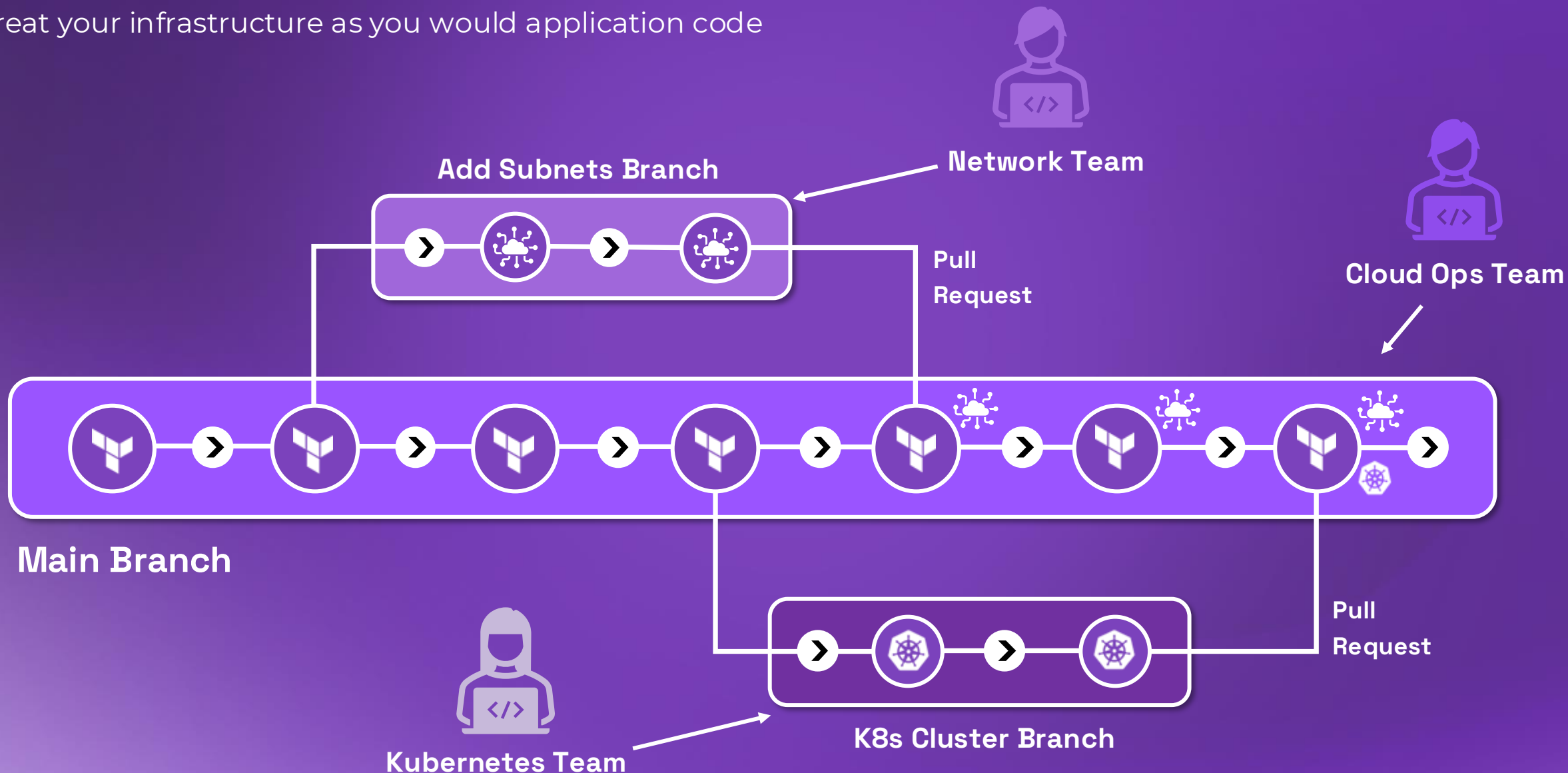
```
1 resource "aws_instance" "basic_server" {  
2   ami                = data.aws_ami.image.id  
3   instance_type      = var.instance_size  
4   subnet_id          = aws_subnet.public_subnets.id  
5   security_groups    = [aws_security_group.ingress-ssh.id]  
6   associate_public_ip_address = true  
7   key_name           = aws_key_pair.generated.key_name  
8 }
```



Data
Application

Infrastructure as Code

Treat your infrastructure as you would application code





Speed & Consistency

Terraform can help you quickly spin up and decommission infrastructure for development, testing, QA, and production.

By defining infrastructure as code, you can reduce the risk of mistakes. You can create standardized components that can be integrated across projects consistently.

You can save infrastructure resources and their dependencies as code in a specified environment.



Core Components



Terraform Core

CLI tool that provisions and manages infrastructure resources as defined in Terraform configuration files



Providers

Extends the functionality of Terraform for specific platforms, such as public cloud providers, SaaS offerings, etc.



Resources

Infrastructure components or services that are managed by Terraform (virtual machines, networks, DNS records, etc.)



State

How Terraform maps the desired configuration with real-world resources on the target platform



Modules

Reusable and sharable blocks of code that can be called over and over again





Terraform Editions



Terraform Community



Command-Line Based



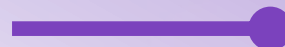
HCP Terraform



SaaS Platform



Terraform Enterprise



Self-Hosted & Managed

Compare Terraform with Other Tools

Terraform is often confused with other popular tools commonly found in many organizations.

Let's set the record straight...



Similar to Terraform



AWS
CloudFormation



Azure
Bicep



Pulumi

Infrastructure as Code Tools

- ❖ Infrastructure Provisioning and Management
 - ❖ Declarative Language
 - ❖ Immutable Infrastructure
 - ❖ Modular Design

Different but Complementary



ANSIBLE



CHEF™



puppet

Configuration Management Tools

- ❖ Installing Packages
 - ❖ Managing Files
- ❖ Manage Configuration Systems
 - ❖ Automating Routine Tasks
 - ❖ Orchestrating Multi-Step Tasks



What Does It Mean to Be Declarative?

A declarative approach focuses on defining *what* you want the outcome to be, not *how* to achieve it.

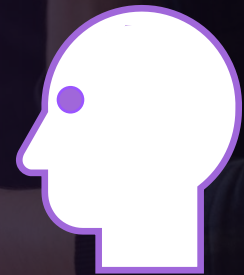


I want an EC2 instance with these properties.

DECLARATIVE

vs

Step 1: Build VM
Step 2: Configure VM
Step 3: Attach to Network



IMPERATIVE

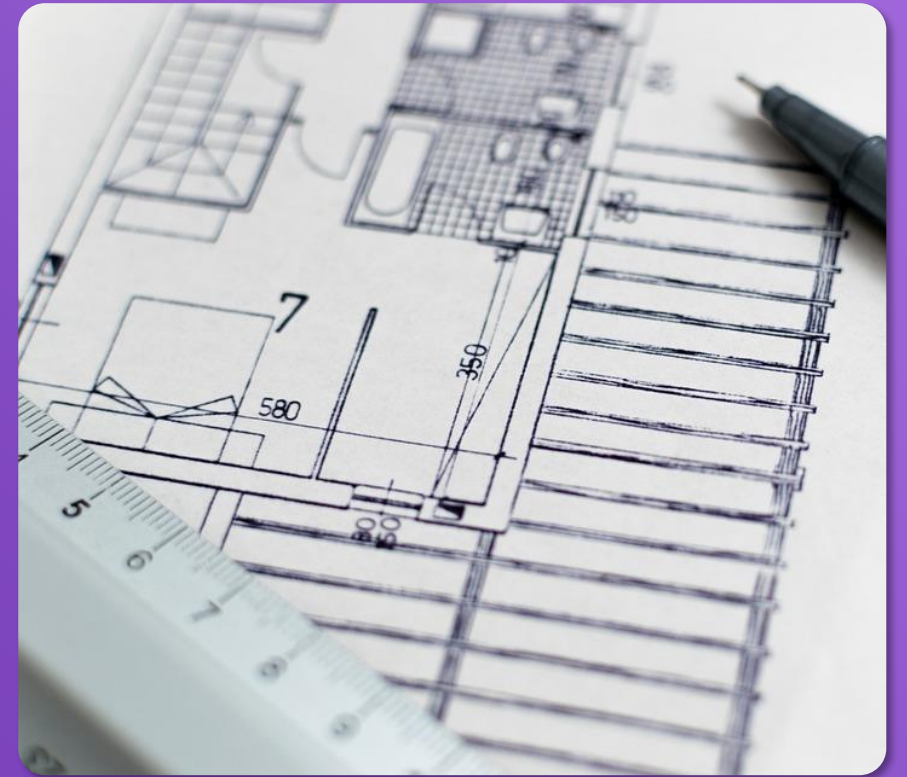


How Terraform Defines Desired State

The desired state is the blueprint of your infrastructure: resources, properties, and relationships.

What you write in your Terraform configuration is known as the desired state. You are telling Terraform "Create resources exactly how I've defined it within my Terraform configuration."

Terraform compares the current state of 'real-world' resources with the desired state and makes the appropriate changes.



How Do I Get Started with Terraform?

1

Download and Install Terraform

Get the proper binary for your machine.

2

Prepare Your Workspace

Install your favorite IDE, create directories, etc.

3

Obtain Credentials for Target Platform

Terraform needs a way to communicate with your cloud

4

Write Your First Terraform Config

Start with basic resources and iterate from there.

SUMMARY



Terraform is a tool created by HashiCorp that is used to manage infrastructure as code.



Terraform is platform-agnostic. It can be used to manage infrastructure on any cloud platform or data center.



Terraform takes a declarative approach, which means that users describe what they want and Terraform will figure out



Benefits of Terraform include version control, scalability, increased collaboration, and repeatability



Terraform uses the HashiCorp Configuration Language (HCL) to define infrastructure



Terraform's core components include Terraform Core, Providers, Resources, State, and Modules.

LECTURE

Learn the Basics of HCL





HashiCorp Configuration Language (HCL)

Terraform configuration language to describe the infrastructure that Terraform manages.



Declarative Language

Used in Terraform (and other HashiCorp products) to define infrastructure configurations



Easy to Read and Write

HCL provides a clean and human-readable syntax for expressing resources and dependencies



Terraform's Primary Interface

Configuration files that you write tell Terraform what infrastructure you want to create or what data to retrieve

HashiCorp Configuration Language (HCL)



```
1
2  block_type "block_label" "block_label" {
3    first_argument = expression or value
4    second_argument = expression or value
5    third          = expression or value
6  }
7
8  attribute_abc = "value_1"
9  attribute_2   = "value_2"
```



HCL Example



data
blocks {

resource
block {

```
main.tf

1  # Retrieve the list of AZs in the current AWS region
2  data "aws_availability_zones" "available" {}
3  data "aws_region" "current" {}
4
5  # Define the VPC
6  resource "aws_vpc" "vpc" {
7      cidr_block = var.vpc_cidr
8
9      tags = {
10         Name      = var.vpc_name
11         Environment = "demo_environment"
12         Terraform  = "true"
13     }
14 }
```

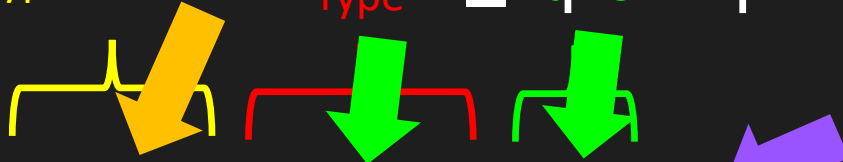
single-line
comment

arguments

Block Example



Type of block Resource type Name



```
1 resource "aws_vpc" "vpc" {
2     cidr_block = var.vpc_cidr
3
4     tags = {
5         Name      = var.vpc_name
6         Environment = "demo_environment"
7         Terraform  = "true"
8     }
9 }
```



Blocks are delineated by { }

All contents within the block are arguments some type of value



Blocks start with a keyword

Defines what type of block it is, such as resource, data, variable, data, etc.



Blocks have named values

Terraform can access information created by or obtained by a block

HCL Style Guide



Writing Terraform code in a consistent style makes it easier to read and maintain.

- Use # for single and multi-line comments
- Use underscores to separate multiple words in names
- Indent two spaces for each nesting level
- Align equal signs for values on consecutive lines
- Use empty lines to separate groups of arguments in a block

```
1 # single-line comment
2 block_type "block_label" "block_label" {
3     first_argument = expression or value
4     second_argument = expression or value
5     third         = expression or value
6 }
7
8 attribute_abc = "value_1"
9 attribute_2   = "value_2"
```



Resource Referencing

Resource referencing allows you to connect resources within your configuration, making it easy to share data between resources and build dependencies.



Create More Dynamic Configurations

Terraform can use properties from one resource in another and avoid hardcoding values, creating a dynamic configuration



Automatic Dependency Mapping

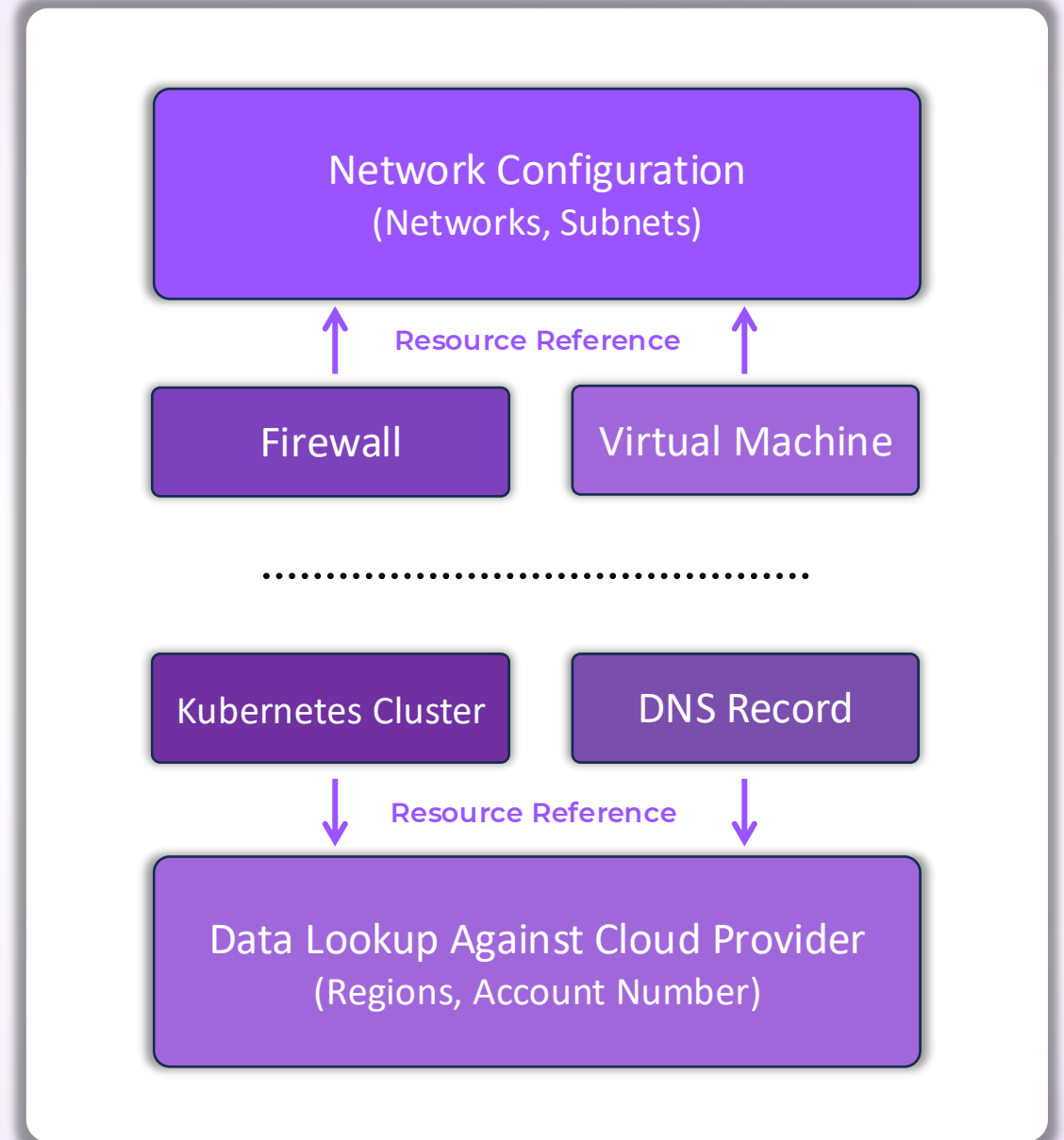
Terraform automatically determines the order of resource creation based on references.



Resource Identifiers

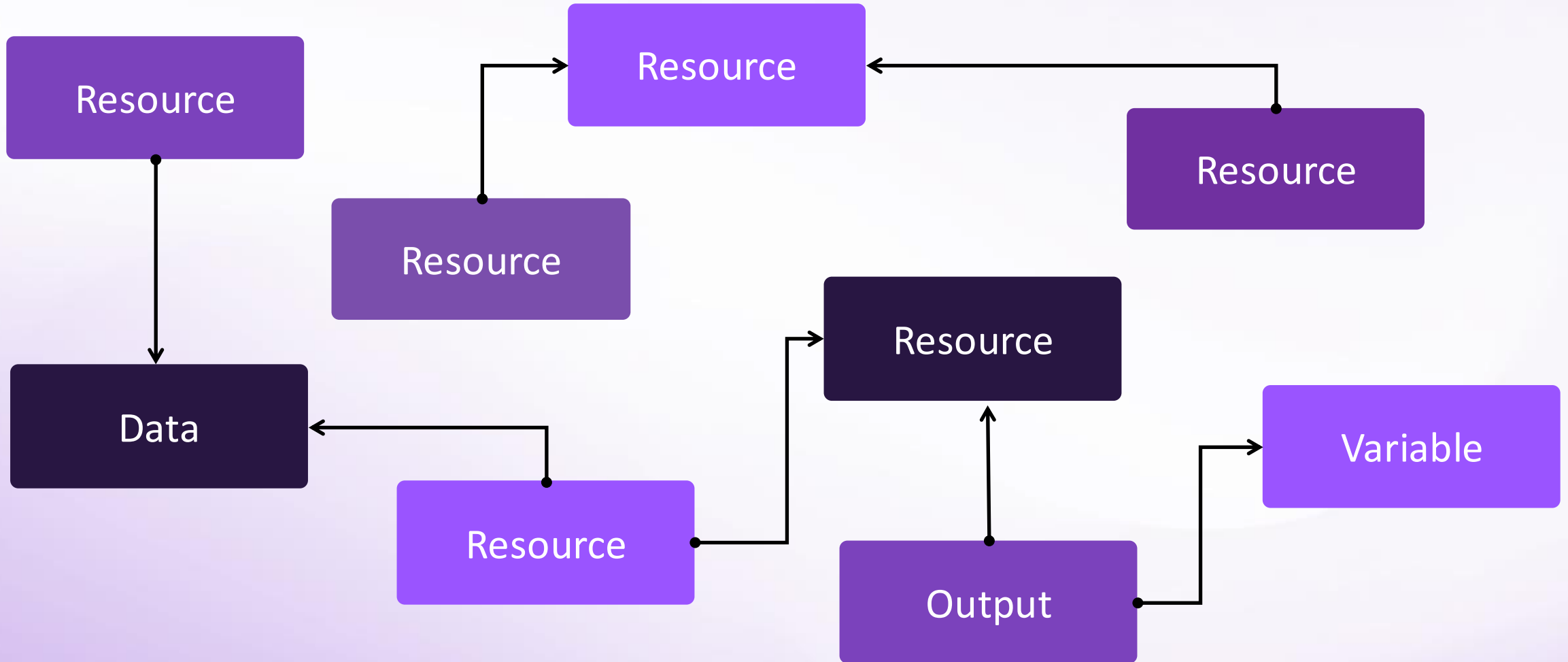
Each Terraform block is identified by a **type** and a **name**, which makes it easy to reference in other parts of your configuration

®Copyright Bryan Krausen – DO NOT DISTRIBUTE



Resource Referencing

The Reality of Terraform



DEMO

Learn the Basics of HCL



Best Practices for HCL



File Extensions

Terraform HCL files use the .tf .tfvars file extension. This allows Terraform and related tools to recognize and apply configurations properly



Formatting

Correctly formatting files improves readability and maintainability. Follow the style guide or use built-in tools to format your files



Organization

Organize functionality within files (variables in a variables file, outputs in an outputs file, etc.



Commenting and Documentation

Include comments and README.md files to provide context and instructions, making it easier for others to understand and use the code



Avoid Hardcoding Values

Use variables for values that may change which makes configurations for flexible and reusable.



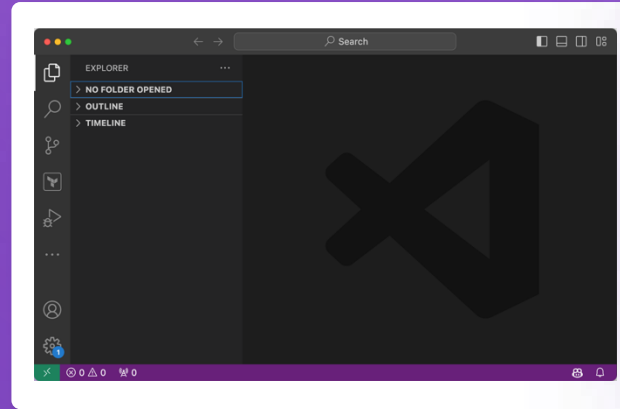


Enhancing Code Quality

Using Linters, Extensions, and Built-In Tools for Code

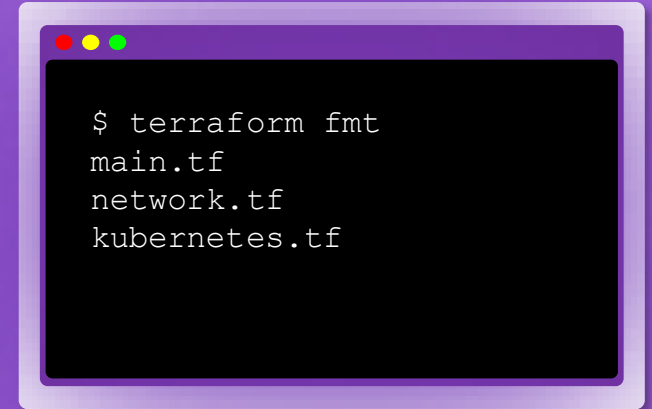
Properly formatted code improves readability and reduces errors. Use a VSCode extension or linter for real-time syntax checks and best-practice suggestions.

Or run `terraform fmt` to automatically format HCL files with a Terraform-related extension (`.tf`)



Linters and Extensions

Linters provide immediate feedback on code issues, while extensions offer additional features like autocompletion and integrated commands



Built-In Tools

`terraform fmt` automatically formats Terraform code to ensure consistency and readability, making collaboration easier across teams.

SUMMARY



HCL is a declarative language that is easy to read and write, and is Terraform's primary interface.



A single Terraform file can have one or many blocks to define your infrastructure



HCL uses blocks for data, resources, etc., and these blocks can be nested.



Use the '.tf' file extension so Terraform and related tools can recognize and apply configurations properly.



Single-line and block comments are used in HCL.



Use linters, extensions, and built-in tools like 'terraform fmt' to enhance code quality.